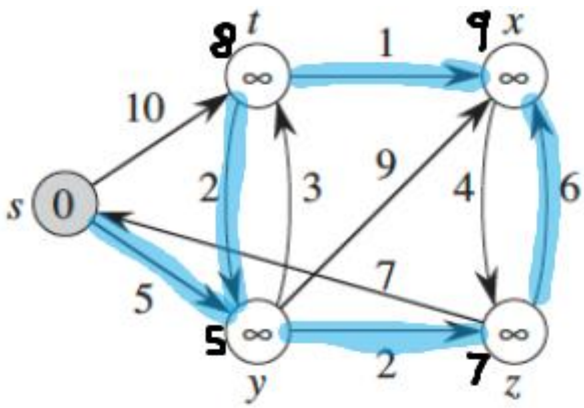
1. Weights 1...|V| : O(E+V)
   Weights 1...W : (constant W): O(E)

2. Weights 1...|V| : O(E+V)
   Weights 1...W : (constant W): O(E)

3. From s:
   dist(s) = 0
   dist(y) = 5 (path: s → y)
   dist(z) = 7 (path: s → z)
   dist(t) = 8 (path: s → y → t)
   dist(x) = 9 (path: s → y → t → x)

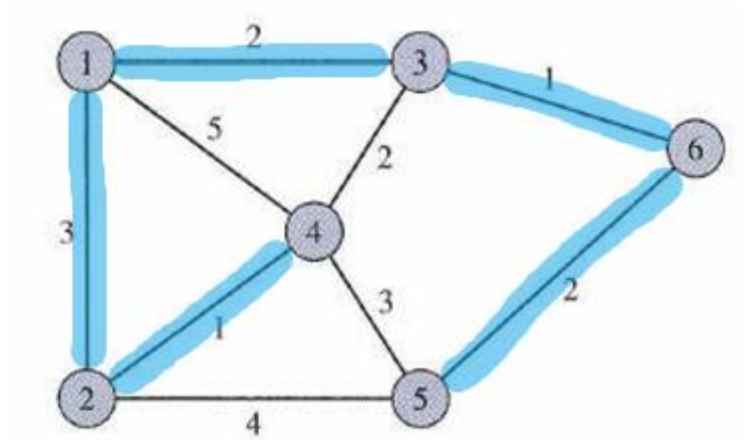
   From z:
   dist(z) = 0
   dist(x) = 6 (path: z → x)
   dist(s) = infinite (unreachable)
   dist(y) = infinite (unreachable)
   dist(t) = infinite (unreachable)



4. Graph :
   A → B (1)
   A → C (10)
   C → B (-20)

   Shortest path A → C → B = -10
   Dijkstra returns B = 1, Dijkstra fails with negative weights

5. Bellman final distances (source = 1)
   Dist(1) = 0
   Dist(2) = 3 (path: 1→2)
   Dist(3) = 2 (path: 1→3)
   Dist(4) = 4 (path: 1 →2 →4)
   Dist(5) = 5 (path: 1→3 →6 →5)
   Dist(6) = 3 (path: 1 →3 → 6)



6. One LCS :
   {0, 1, 0, 1, 0}

7. A) union by size
   0 → 8
   1 → 3
   2 → 1
   3 → 3
   4 → 3
   5 → 3
   6 → 6
   7 → 1
   8 → 3
   9 → 0
   10 → 8

11 → 11
12 → 3

b) union by height
0 → 8
1 → 1
2 → 1
3 → 1
4 → 3
5 → 3
6 → 6
7 → 1
8 → 1
9 → 0
10 → 8
11 → 11
12 → 3

c) Union by size + path compression
0 → 3
1 → 3
2 → 3
3 → 3
4 → 3
5 → 3
6 → 6
7 → 3
8 → 3
9 → 3
10 → 3
11 → 11
12 → 3

8. Number of trees of height h = 1

9. A) check if ci and cj are in the same province
   Find(ci) == Find(cj)

b)pseudocode for grouping provinces

```
MakeSet(c1...cn)

for i = 1 to n:
  for j = 1 to n:
     if R(ci, cj) == 1:
        Union(ci, cj)
```